

PYTHON을 이용한 데이터 분석 · 12강

머신러닝으로 문제 풀기

이론편 — Titanic · Iris · 고객 구매 예측으로 배우는 분류 모델



오늘의 학습 목표



머신러닝 프로세스 이해

데이터 준비 → 모델 선택 → 학습 → 평가의 표준 흐름 익히기



Train-Test Split

학습 데이터와 테스트 데이터를 나누는 이유와 방법



여러 분류 모델 비교

KNN · Decision Tree · Logistic Regression을 직접 써보고 비교



모델 평가와 해석

정확도와 혼동행렬로 성능을 확인하고 결과를 설명하기

머신러닝이란?

정의 과거 데이터로부터 **패턴을 학습**해서 새로운 데이터에 대한 **예측**을 수행하는 기술입니다.

오늘 우리가 풀 3가지 문제



Titanic

이 승객이 생존했을까?



Iris

이 꽃의 품종은?



고객 구매

이 고객이 구매할까?

머신러닝의 종류 — 우리는 지도학습

지도학습 (오늘)

입력 X (특성) + y (정답 레이블)

회귀 9강 — 연속적인 값 예측

분류 10강·오늘 — 카테고리 예측

목표 $X \rightarrow y$ 매핑 함수 찾기

비지도학습 (11강)

입력 X (특성만, 정답 없음)

클러스터링 11강 — 비슷한 것끼리 묶기

예시 고객 세분화

목표 X 안의 자연스러운 패턴 발견

오늘의 초점 지도학습 중에서도 '분류(Classification)' — Yes/No, A/B/C 중 하나를 예측하는 문제입니다.

머신러닝의 표준 프로세스

1

데이터 준비



2

모델 선택



3

모델 학습



4

예측 + 평가

탐색 + 전처리 (결측치, 인코딩) KNN, Decision Tree, Logistic 등 fit() — 과거 데이터로 패턴 학습 predict() + accuracy 확인

핵심 이 4단계는 어떤 데이터, 어떤 모델을 쓰든 동일합니다. Titanic이든 Iris든 고객 데이터든 같은 프로세스를 반복할 뿐입니다.

sklearn 문법 `model.fit(X_train, y_train)` → `model.predict(X_test)` — 9강·10강·11강
과 완전히 같은 패턴!

왜 데이터를 나눠야 할까?

문제 모델에게 시험 문제와 정답을 모두 외우게 하면, 실제 시험(새로운 데이터)에서는 성능을 알 수 없습니다.

Training Data (70%)

Test (30%)

Training Data

모델에게 '정답이 뭐야' 하고 보여주며 학습시키는 데이터

Test Data

모델이 본 적 없는 데이터로 '진짜 실력'을 확인하는 데이터

```
train_test_split(X, y, test_size=0.3, random_state=42)
```

오늘 사용할 3가지 분류 모델

KNN

K-Nearest Neighbors

가장 가까운 K명의 이웃을 보고 다
수결로 결정

Decision Tree

의사결정트리 (10강 복습)

질문을 반복해서 답을 찾아가는 스
무고개 방식

Logistic Regression

로지스틱 회귀

확률을 계산해서 더 높은 쪽으로
분류

EVALUATION

모델을 평가하는 방법

정확도 (Accuracy)

전체 예측 중 몇 %를 맞혔는가 → $\text{accuracy} = \frac{\text{맞은 개수}}{\text{전체 개수}}$

혼동행렬 (Confusion Matrix)

정답/오답을 카테고리별로 나눠서 자세히 확인

	예측: 생존	예측: 사망
실제: 생존	True Positive (정확히 맞음)	False Negative (놓침)
실제: 사망	False Positive (오판)	True Negative (정확히 맞음)

생존자가 1%뿐인 데이터라면, '무조건 사망'이라 답해도 99% 정확도! 정확도만 보면 안 되는 이유.

파이썬에서는 어떻게 할까? — 다음 실습 예고

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(accuracy_score(y_test, y_pred))
```

train_test_split

데이터를 학습/테스트로 분리

model.fit()

학습 데이터로 패턴 학습

model.predict()

새 데이터에 대한 예측 수행

현실에서는 어떻게 쓰일까?



은행

대출 승인/거절 예측 — 신용도 데이터를 기반으로 분류



병원

질병 진단 — 검사 수치를 기반으로 양성/음성 분류



쇼핑몰

고객이 뭘 좋아할지 추천 — 오늘 실습한 것과 동일한 원리



이메일

스팸 필터 — 이 메일이 스팸인지 아닌지 분류

실습 준비물

필요한 라이브러리

- pandas, numpy
- matplotlib, seaborn
- scikit-learn (sklearn)

사용할 데이터셋

titanic.csv

890명, 생존 예측

iris_for_training.csv

150개, 품종 분류

customer_purchase.csv

300명, 구매 예측

Q & A

질문 있으신가요?



머신러닝이 뭔지 감이 왔나요?

자, 이제 실제로 코드를 짜면서 배워봅시다!

SUMMARY

오늘의 핵심 정리

- ✓ 머신러닝 프로세스 데이터 준비 → 모델 선택 → 학습 → 평가의 4단계
- ✓ Train-Test Split 본 적 없는 데이터로 테스트해야 진짜 성능을 안다
- ✓ 3가지 분류 모델 KNN · Decision Tree · Logistic Regression 비교
- ✓ 정확도와 혼동행렬 정확도만으로는 부족, 불균형 데이터 주의
- ✓ 같은 프로세스, 다른 문제 Titanic · Iris · 고객 구매 — 프로세스는 하나

NEXT 12강 실습 — Titanic·Iris·고객 데이터로 3가지 모델 직접 구현

PYTHON을 이용한 데이터 분석 · 12강 실습

파이썬으로 분류 모델 3종 직접 구현하기

```
model =  
KNeighborsClassifier()  
model.fit(X_train, y_train)  
Accuracy = 0.79
```

실습편 — Titanic · Iris · 고객 구매 데이터로 KNN, Decision Tree, Logistic Regression 비교

환경 설정 체크리스트

1 Import 확인

```
pandas / sklearn.model_selection / neighbors / tree / linear_model / metrics
```

2 Titanic 로드

```
titanic = pd.read_csv('titanic.csv')
```

3 Iris 로드

```
iris = pd.read_csv('iris_for_training.csv')
```

4 고객 데이터 로드

```
customer = pd.read_csv('customer_purchase.csv')
```

5 구조 확인

각 데이터셋에서 `df.head()` / `df.info()` 실행

라이브러리 импорт 및 데이터 로드

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

titanic = pd.read_csv('titanic.csv')
print(f'승객 수: {len(titanic)}명')
```


Titanic 데이터 탐색 — 무엇을 예측할까?



1912년 침몰한 타이타닉호. 890명의 승객 정보로 '생존 여부'를 예측합니다.

주요 컬럼

Survived	생존(1)/사망(0) — 정답
Pclass	객실 등급 (1/2/3)
Sex	성별
Age	나이
Fare	탑승료

탐색 순서

1. `df.head()` — 첫 5행 확인
2. `df.info()` — 데이터 타입/결측치
3. `df.describe()` — 통계 요약
4. 생존율 계산
5. 성별별 생존율 비교

PART 1

탐색 코드 및 성별별 생존율

```
print(titanic.head())
print(titanic.info())
print(titanic.describe())

# 생존율 계산
print(f'생존율: {titanic["Survived"].mean()*100:.1f}%')

# 성별별 생존율 비교
print(titanic.groupby('Sex')['Survived'].mean())

# 결과 예: female 0.74 / male 0.19
# ♀ 여성의 생존율이 남성보다 훨씬 높다!
```

데이터 전처리 — 왜 필요할까?

⚠ 머신러닝 모델은 숫자만 이해합니다. 문자와 결측치를 처리해야 합니다!

문제 1: 문자 데이터

Sex 컬럼이 'male' / 'female' 문자로 되어 있음

→ 숫자로 인코딩: male=1, female=0

문제 2: 결측치

Age 일부가 비어있음 (177개 결측)

```
# 성별 인코딩
titanic['Sex'] = (
    titanic['Sex'] == 'male'
).astype(int)
```

```
# 결측치를 평균으로 채우기
X['Age'].fillna(
    X['Age'].mean(),
    inplace=True)
```

```
# ✓ 이제 모델이 이해 가능
```

특성 선택 및 전처리 전체 코드

```
# 성별 인코딩 (male=1, female=0)
titanic['Sex'] = (titanic['Sex'] == 'male').astype(int)

# 특성(X)과 타겟(y) 분리
X = titanic[['Pclass', 'Sex', 'Age', 'Fare']].copy()
y = titanic['Survived']

# 결측치 처리
X['Age'].fillna(X['Age'].mean(), inplace=True)

print(f'특성 형태: {X.shape}')
print(f'결측치: {X.isnull().sum().sum()}개') # 0이어야 함
```

Train-Test Split — 70:30으로 나누기

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.3, random_state=42)  
  
print(f'학습 데이터: {len(X_train)}명') # 약 623명  
print(f'테스트 데이터: {len(X_test)}명') # 약 267명
```

왜 random_state=42? 같은 숫자를 쓰면 매번 같은 방식으로 데이터가 나뉩니다. 결과를 재현하고 비교하기 위해 고정합니다.

3가지 모델 한 번에 학습하기

```
# 1. KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 2. Decision Tree (10강 복습!)
tree = DecisionTreeClassifier(random_state=42, max_depth=5)
tree.fit(X_train, y_train)

# 3. Logistic Regression
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train, y_train)

# 세 모델 모두 fit() 패턴이 동일! 예측은 .predict(X_test)
```

모델 평가 — 정확도 비교

```
y_pred_knn = knn.predict(X_test)
y_pred_tree = tree.predict(X_test)
y_pred_lr = lr.predict(X_test)

acc_knn = accuracy_score(
    y_test, y_pred_knn)
acc_tree = accuracy_score(
    y_test, y_pred_tree)
acc_lr = accuracy_score(
    y_test, y_pred_lr)

print(f'KNN: {acc_knn:.2%}')
```

예상 정확도 범위

KNN

65 ~ 75%

Decision Tree

70 ~ 80%

Logistic Regression

72 ~ 82%

💡 85% 이상이면 과적합을 의심해보세요!

혼동행렬로 자세히 들여다보기

```
cm = confusion_matrix(y_test, y_pred_tree)
print(cm)

print(f'정확히 예측: {cm[0,0] + cm[1,1]}명')
print(f'틀린 예측: {cm[0,1] + cm[1,0]}명')
```

생각해보기 타이타닉에서 '생존자를 사망으로 잘못 예측'하는 것과 '사망자를 생존으로 잘못 예측'하는 것, 어느 쪽이 더 심각한 오류일까요?

모델 성능 시각화

```
models = ['KNN', 'Decision Tree', 'Logistic Regression']
accuracies = [acc_knn, acc_tree, acc_lr]

plt.figure(figsize=(10, 6))
bars = plt.bar(models, accuracies,
               color=['#FF6B6B', '#4ECDC4', '#45B7D1'])
plt.ylabel('정확도')
plt.title('Titanic 데이터셋 - 모델 성능 비교')
plt.ylim([0, 1])

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + 0.5, bar.get_height(), f'{acc:.1%}')
plt.show()
```

Iris 데이터 — 같은 프로세스, 다른 문제



붓꽃의 꽃잎/꽃받침 길이로 3가지 품종(Setosa/Versicolor/Virginica) 중 하나를 예측

```
iris = pd.read_csv('iris_for_training.csv')
X_iris = iris.iloc[:, :-1]    # 4개 특성
y_iris = iris['Species']      # 품종 (0, 1, 2)

X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42)

knn_iris = KNeighborsClassifier(n_neighbors=5)
knn_iris.fit(X_train_i, y_train_i)
acc = accuracy_score(y_test_i, knn_iris.predict(X_test_i))
print(f'Iris 정확도: {acc:.2%}') # 보통 90% 이상!
```

고객 구매 데이터 — 실무형 문제



나이, 소득, 구매 경향, 충성도 등으로 '구매 여부'를 예측 — 정확도는 Iris보다 낮을 수 있음

```
customer = pd.read_csv('customer_purchase.csv')
X_c = customer.drop(['고객_ID', '구매'], axis=1)
y_c = customer['구매']

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_c, y_c, test_size=0.3, random_state=42)

tree_c = DecisionTreeClassifier(random_state=42)
tree_c.fit(X_train_c, y_train_c)
acc_c = accuracy_score(y_test_c, tree_c.predict(X_test_c))
print(f'고객 구매 예측 정확도: {acc_c:.2%}')
```

CHALLENGE 1

3개 데이터셋 × 3개 모델 종합 비교

TODO: Titanic·Iris·고객 데이터 각각에서 KNN·Decision Tree·Logistic Regression 정확도를 모두 계산하고, 하나의 표로 정리하세요.

```
results = pd.DataFrame({
    '데이터셋': ['Titanic', 'Iris', '고객 구매'],
    'KNN': [acc_knn, acc_knn_iris, acc_knn_cust],
    'Decision Tree': [acc_tree, acc_tree_iris, acc_tree_cust],
    'Logistic Regression': [acc_lr, acc_lr_iris, acc_lr_cust]
})
print(results)
```

왜 데이터마다 성능이 다를까?

TODO: 결과표를 보고 아래 질문에 답해보세요.

Q1. Iris는 왜 모든 모델이 90% 이상으로 잘 작동할까?

Q2. 고객 구매 데이터는 왜 상대적으로 정확도가 낮을까?

Q3. Titanic에서 Decision Tree가 KNN보다 나은 이유는?

Q4. 만약 age 컬럼이 없다면 정확도는 어떻게 변할까?

CHALLENGE 3

하이퍼파라미터 바꿔보기

모델의 설정값(하이퍼파라미터)을 바꾸면 성능이 어떻게 달라질까요?

```
# n_neighbors 값을 바꿔보기
for k in [1, 3, 5, 10, 20]:
    knn = KNeighborsClassifier(
        n_neighbors=k)
    knn.fit(X_train, y_train)
    acc = accuracy_score(
        y_test,
        knn.predict(X_test))
    print(f'k={k}: {acc:.2%}')
```

생각해보기:

k가 너무 작으면? → 노이즈에 민감 (과적합 위험)

k가 너무 크면? → 너무 단순해짐 (과소적합 위험)

→ 다음 13강에서 이 '최적값 찾기'를 자동화하는 법을 배웁니다!

실습 완료 확인 사항

- | | | |
|---|------------------|--|
| 1 | Titanic 데이터 탐색 | df.head(), describe(), 성별별 생존율 확인 |
| 2 | 전처리 | 성별 인코딩, 결측치 처리 완료 |
| 3 | Train-Test Split | 70:30으로 데이터 분할 |
| 4 | 3개 모델 학습 | KNN, Decision Tree, Logistic Regression 모두 fit() |
| 5 | 모델 평가 | 정확도 및 혼동행렬 확인 |
| 6 | Iris 적용 | 동일 프로세스로 품종 분류 수행 |
| 7 | 고객 데이터 적용 | 동일 프로세스로 구매 예측 수행 |
| 8 | 도전 문제 | 종합 비교표 작성 및 질문 답변 |

오늘의 핵심

- Part 0** → 라이브러리 импорт 및 3개 데이터셋 로드
- Part 1** → Titanic 데이터 탐색 및 생존 패턴 파악
- Part 2** → 성별 인코딩, 결측치 처리로 전처리 완료
- Part 3** → Train-Test Split 후 KNN·Tree·Logistic 학습
- Part 4** → 정확도와 혼동행렬로 모델 성능 평가
- Part 5** → 같은 프로세스를 Iris·고객 데이터에 반복 적용